

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



THIẾT KẾ VI MẠCH (CO3098)

Bài Thực Hành 4

RTL DESIGN & SIMULATION OF SEQUENTIAL CIRCUITS USING XCELIUM

Lớp: L04 - Nhóm: 08 - Học Kỳ HK252
Ngày nộp: 10/05/2026

GVHD: Thầy Phạm Kiều Nhật Anh

Sinh Viên	MSSV
Phạm Công Võ	2313946
Phạm Minh Đức	2310797
Nguyễn Trần Như Phước	2212719
Nguyễn Thanh Tùng	2112711

TP.Hồ Chí Minh, Tháng 03/2026



Mục lục

1	Introduction	2
1.1	Mục tiêu Lab	2
1.2	Công cụ sử dụng	2
1.3	Tổng quan thiết kế - Ring Flasher	2
2	Design Specification	3
2.1	Mô tả chức năng	3
2.2	Interface - Bảng tín hiệu	4
2.3	Máy trạng thái (FSM)	5
3	RTL Implementation	6
3.1	Cấu trúc thiết kế	6
3.2	RTL Code	7
4	Testbench	9
4.1	Tổng quan Testbench	9
4.2	Testbench Code	10
4.3	Chi tiết từng Test Case	12
5	Kết luận	16
5.1	Kết quả đạt được	16
5.2	Những điểm đã học được	16

1 Introduction

1.1 Mục tiêu Lab

Lab 4 nhằm trang bị cho sinh viên nền tảng thực hành trong quy trình thiết kế, hiện thực và xác minh mạch số tuần tự ở mức RTL (*Register Transfer Level*). Cụ thể, sau khi hoàn thành Lab, sinh viên có khả năng:

- Hiện thực **Finite State Machine (FSM)** bằng Verilog RTL với nhiều trạng thái và điều kiện chuyển trạng thái phức tạp, bao gồm cơ chế *toggle mode* theo pha.
- Xây dựng môi trường mô phỏng RTL chuẩn công nghiệp trên nền tảng **Cadence Xcelium**, bao gồm biên dịch, chạy simulation và xuất waveform từ command-line.
- Viết **testbench** bằng Verilog với nhiều test case độc lập, bao phủ các tình huống hoạt động bình thường, restart và reset bất thường.
- Phân tích **waveform** sử dụng **SimVision** để xác minh hành vi FSM theo từng clock cycle và debug logic.

1.2 Công cụ sử dụng

Công cụ	Phiên bản	Vai trò
Cadence Xcelium	2021	Trình biên dịch và thực thi mô phỏng HDL. Hỗ trợ Verilog/SystemVerilog, cho phép chạy simulation từ command-line với lệnh <code>xrun</code> .
SimVision	2021	Giao diện đồ họa tích hợp để xem waveform, duyệt source code (<i>Source Browser</i>) và trace logic qua <i>Signal Tracer</i> .

Bảng 1: Công cụ sử dụng trong Lab 4

1.3 Tổng quan thiết kế - Ring Flasher

Thiết kế trong Lab 4 là module **Ring Flasher** - một mạch điều khiển tuần tự điều khiển dãy 16 đèn LED (`led_out[15:0]`) theo một chuỗi quét có điều kiện, được hiện thực bằng kiến trúc **Moore FSM 3 trạng thái** trong Verilog RTL.

Dự án bao gồm hai thành phần chính:

File	Vai trò
<code>ring_flasher.v</code>	RTL design - mô tả hành vi phần cứng của module Ring Flasher theo kiến trúc Moore FSM với 3 trạng thái: IDLE, UP, DOWN.
<code>testbench_ring_flasher.v</code>	Testbench - tạo môi trường mô phỏng với 4 test case kiểm tra toàn diện các kịch bản hoạt động của thiết kế.

Bảng 2: Các file thiết kế trong Lab 4

2 Design Specification

2.1 Mô tả chức năng

Module **Ring Flasher** hiện thực hệ thống điều khiển 16 đèn LED (`led_out[15:0]`) hoạt động theo chuỗi quét tuần hoàn có điều kiện, dựa trên ba tín hiệu đầu vào: `clk`, `rst_n` và `start_sig`.

Tín hiệu điều khiển:

- **Clock** (`clk`): Xung nhịp hệ thống. Toàn bộ logic tuần tự - cập nhật trạng thái, con trỏ `ptr` và các bộ đếm - được đồng bộ và thực thi tại **cạnh lên** (*positive edge*).
- **Reset** (`rst_n`): Reset **bất đồng bộ tích cực thấp** - có hiệu lực **ngay lập tức**, không chờ cạnh clock.
 - `rst_n = 0`: FSM lập tức chuyển về IDLE, toàn bộ LED tắt (`led_out = 0`), các thanh ghi nội `ptr`, `step_cnt`, `phase_cnt` đều được reset về 0.
 - `rst_n = 1`: Hệ thống hoạt động bình thường.
- **Start Signal** (`start_sig`): Tín hiệu kích hoạt; FSM chỉ **sample** tín hiệu này tại đúng **hai thời điểm** xác định trong chu trình hoạt động:
 - Khi FSM đang ở IDLE: `start_sig = 1` kích hoạt bắt đầu một vòng quét mới, FSM chuyển sang trạng thái UP.
 - Khi vòng quét kết thúc (*all OFF*): `start_sig = 1` restart vòng mới (reset `ptr`, `phase_cnt` về 0); `start_sig = 0` đưa FSM về IDLE, dừng hoàn toàn.

Ở tất cả các thời điểm khác, tín hiệu này bị FSM bỏ qua hoàn toàn.

Chuỗi hoạt động trong một vòng quét:

Mỗi vòng quét gồm **12 phase** luân phiên UP/DOWN (6 cặp), chia thành hai giai đoạn dựa trên bộ đếm `phase_cnt`:

Giai đoạn	Điều kiện	Phase	Hành vi tại mỗi bước
Normal mode	<code>phase_cnt < 6</code>	UP (8 bước)	Bật LED tại <code>ptr</code> : <code>led_out[ptr] = 1</code> , sau đó tăng <code>ptr</code> . Kết thúc khi <code>step_cnt = 7</code> .
		DOWN (4 bước)	Tắt LED tại <code>ptr</code> : <code>led_out[ptr] = 0</code> , sau đó giảm <code>ptr</code> . Kết thúc khi <code>step_cnt = 3</code> .
Toggle mode	<code>phase_cnt ≥ 6</code>	UP (8 bước)	Toggle OFF LED tại <code>ptr</code> : <code>led_out[ptr] = ~led_out[ptr]</code> , sau đó tăng <code>ptr</code> .
		DOWN (4 bước)	Toggle ON LED tại <code>ptr</code> : <code>led_out[ptr] = ~led_out[ptr]</code> , sau đó giảm <code>ptr</code> . Số LED sáng giảm dần qua các phase.

Bảng 3: Hành vi FSM theo từng giai đoạn và phase trong một vòng quét

Sau mỗi phase hoàn thành, `phase_cnt` tăng 1 và FSM chuyển sang phase tiếp theo. Khi **all OFF** (`led_out` = 0) được phát hiện tại cuối một phase DOWN trong toggle mode, một vòng quét hoàn tất.

Điều kiện kết thúc vòng (all OFF):

Trạng thái *all OFF* được kiểm tra tại cuối phase DOWN trong toggle mode. Tại thời điểm này, FSM sample `start_sig` để quyết định:

- `start_sig` = 1: FSM **restart** ngay lập tức - reset `ptr` = 0, `step_cnt` = 0, `phase_cnt` = 0, chuyển thẳng sang UP để bắt đầu vòng mới mà không qua IDLE.
- `start_sig` = 0: FSM chuyển về IDLE, dừng hoàn toàn và chờ tín hiệu kích hoạt mới.

2.2 Interface - Bảng tín hiệu

Module `ring_flasher` có giao tiếp tối giản với 3 tín hiệu đầu vào và 1 tín hiệu đầu ra. Bảng 4 mô tả chi tiết hướng, độ rộng bit và chức năng của từng tín hiệu.

Tín hiệu	Hướng	Độ rộng	Mô tả chi tiết
<code>clk</code>	Input	1-bit	Xung nhịp đồng hồ hệ thống. Toàn bộ logic tuần tự - cập nhật trạng thái, con trỏ <code>ptr</code> , bộ đếm - được đồng bộ và thực thi tại cạnh lên (<i>posedge clk</i>).
<code>rst_n</code>	Input	1-bit	Reset bất đồng bộ tích cực thấp . Khi <code>rst_n</code> = 0, FSM lập tức về IDLE, tắt toàn bộ LED và reset các thanh ghi nội (<code>ptr</code> , <code>step_cnt</code> , <code>phase_cnt</code>) về 0, không phụ thuộc vào trạng thái clock.
<code>start_sig</code>	Input	1-bit	Tín hiệu kích hoạt. FSM chỉ sample tín hiệu này tại hai thời điểm xác định: (1) khi đang ở IDLE để khởi động vòng quét mới; (2) khi phát hiện <i>all OFF</i> để quyết định restart hay về IDLE. Ở các thời điểm khác, tín hiệu bị bỏ qua.
<code>led_out[15:0]</code>	Output	16-bit	Thanh ghi đầu ra phản ánh trạng thái tức thời của 16 đèn LED. <code>led_out[i]</code> = 1 nghĩa là LED thứ <i>i</i> đang sáng; = 0 là tắt. Giá trị cập nhật đồng bộ tại <i>posedge clk</i> .

Bảng 4: Bảng tín hiệu giao tiếp của module `ring_flasher`

2.3 Máy trạng thái (FSM)

Module `ring_flasher` được hiện thực theo kiến trúc **Moore FSM** — đầu ra (`led_out`) chỉ phụ thuộc vào trạng thái hiện tại, không phụ thuộc trực tiếp vào đầu vào. FSM gồm **3 trạng thái** được mã hóa 2-bit, trình bày trong Bảng 5.

Trạng thái	Mã	Mô tả
IDLE	2'd0	Trạng thái chờ. Toàn bộ LED tắt, <code>ptr = 0</code> , <code>step_cnt = 0</code> , <code>phase_cnt = 0</code> . FSM giữ nguyên cho đến khi nhận <code>start_sig = 1</code> .
UP	2'd1	Trạng thái quét xuôi, thực hiện 8 bước liên tiếp (<code>step_cnt</code> : 0→7), tăng <code>ptr</code> sau mỗi bước. Ở Normal mode: bật LED; ở Toggle mode: toggle OFF LED tại <code>ptr</code> .
DOWN	2'd2	Trạng thái quét ngược, thực hiện 4 bước liên tiếp (<code>step_cnt</code> : 0→3), giảm <code>ptr</code> sau mỗi bước. Ở Normal mode: tắt LED; ở Toggle mode: toggle ON LED tại <code>ptr</code> . Đây là trạng thái duy nhất kiểm tra điều kiện <i>all OFF</i> .

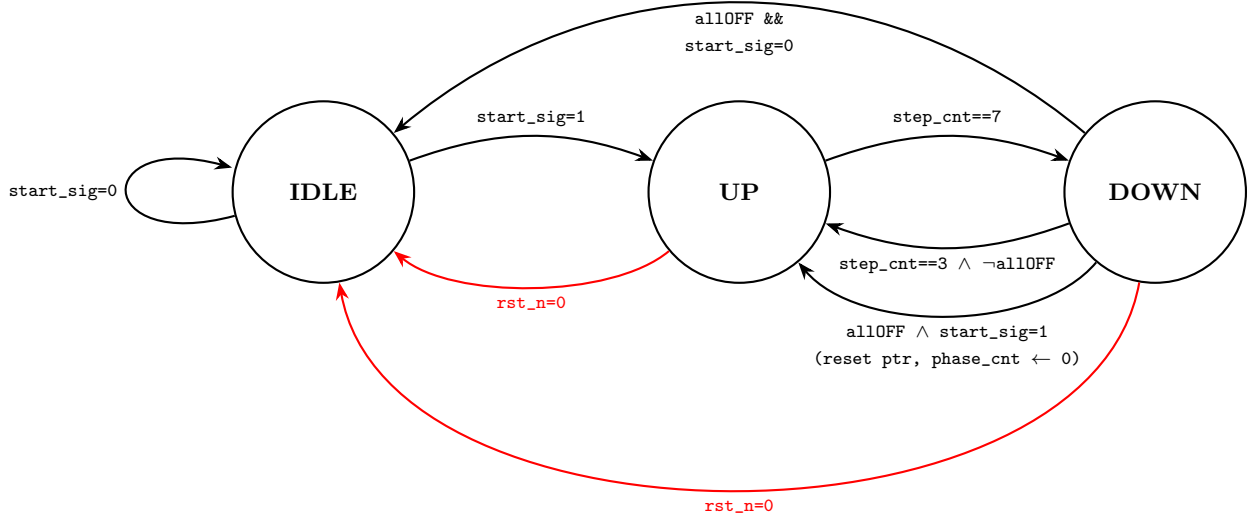
Bảng 5: Bảng trạng thái FSM của module `ring_flasher`

Các điều kiện chuyển trạng thái được tổng hợp trong Bảng 6. Đáng chú ý là chuyển tiếp DOWN→UP có hai nhánh phân biệt: nhánh thông thường (chưa *all OFF*) và nhánh restart (đã *all OFF* nhưng `start_sig = 1`); trong trường hợp restart, `ptr` và `phase_cnt` được reset về 0 trước khi vào UP.

Từ	Đến	Điều kiện chuyển
IDLE	UP	<code>start_sig = 1</code> : kích hoạt bắt đầu vòng quét mới.
UP	DOWN	<code>step_cnt == 7</code> : hoàn thành đủ 8 bước quét xuôi.
DOWN	UP	<code>step_cnt == 3</code> và chưa <i>all OFF</i> : hoàn thành 4 bước quét ngược, tiếp tục phase mới.
DOWN	UP	<i>all OFF</i> và <code>start_sig = 1</code> : vòng kết thúc, restart ngay — reset <code>ptr = 0</code> , <code>phase_cnt = 0</code> rồi vào UP.
DOWN	IDLE	<i>all OFF</i> và <code>start_sig = 0</code> : vòng kết thúc, dừng hoàn toàn.
any	IDLE	<code>rst_n = 0</code> : reset bất đồng bộ, có hiệu lực tức thì bất kể trạng thái hay bước hiện tại.

Bảng 6: Bảng điều kiện chuyển trạng thái FSM

Sơ đồ trạng thái tổng quát của module được minh họa trong Hình 1, thể hiện đầy đủ các nhánh chuyển tiếp và điều kiện kích hoạt tương ứng.



Hình 1: Sơ đồ máy trạng thái module *ring_flasher*

3 RTL Implementation

3.1 Cấu trúc thiết kế

Module *ring_flasher* được hiện thực theo phong cách **single always block** — toàn bộ logic FSM (cập nhật trạng thái, điều khiển con trỏ *ptr*, bộ đếm bước *step_cnt*, bộ đếm pha *phase_cnt* và đầu ra *led_out*) được đặt trong một khối **always** đồng bộ duy nhất với reset bất đồng bộ:

```
always @(posedge clk or negedge rst_n)
```

Cách tổ chức này đảm bảo không phát sinh latch không mong muốn (*latch inference*), đồng thời giữ cho luồng điều khiển rõ ràng, dễ đọc và dễ tổng hợp (*synthesis-friendly*).

Tín hiệu tổ hợp *toggle_mode* được suy ra trực tiếp từ *phase_cnt* bằng một wire đơn giản, không cần thêm trạng thái FSM mới:

```
wire toggle_mode = (phase_cnt >= 4'd6);
```

Khi *toggle_mode* = 0 (Normal mode), UP bật LED và DOWN tắt LED. Khi *toggle_mode* = 1 (Toggle mode), cả UP và DOWN đều thực hiện thao tác đảo bit ($\sim \text{led_out}[\text{ptr}]$), dần dần tắt toàn bộ LED qua nhiều phase.

Bảng 7 tổng hợp các thanh ghi nội của module, bao gồm độ rộng bit và vai trò trong hệ thống:

Thanh ghi	Độ rộng	Vai trò
state	2-bit	Trạng thái FSM hiện tại, mã hóa 2-bit: IDLE=0, UP=1, DOWN=2.
ptr	4-bit	Con trỏ chỉ vị trí LED đang được thao tác (0–15). Tăng trong UP, giảm trong DOWN.
step_cnt	3-bit	Bộ đếm bước trong phase hiện tại. UP: đếm 0→7 (8 bước); DOWN: đếm 0→3 (4 bước). Reset về 0 khi chuyển phase.
phase_cnt	4-bit	Số phase đã hoàn thành trong vòng quét hiện tại (0–11). Quyết định Normal mode hay Toggle mode qua toggle_mode. Reset về 0 khi restart hoặc về IDLE.
led_out	16-bit	Thanh ghi đầu ra phản ánh trạng thái tức thời của 16 đèn LED. Cập nhật đồng bộ theo posedge clk.

Bảng 7: Các thanh ghi nội trong module *ring_flasher*

3.2 RTL Code

Toàn bộ hiện thực RTL được trình bày trong Listing ???. Module gồm ba phần chính: khối reset bất đồng bộ khởi tạo toàn bộ thanh ghi về 0; khối case xử lý logic từng trạng thái; và nhánh default đảm bảo FSM luôn có trạng thái hợp lệ, tránh treo (*deadlock*) khi gặp trạng thái không xác định.

```
1  `timescale 1ns / 1ps
2  module ring_flasher(
3      input  wire      clk,
4      input  wire      rst_n,
5      input  wire      start_sig,
6      output reg  [15:0] led_out
7  );
8      localparam IDLE = 2'd0;
9      localparam UP   = 2'd1;
10     localparam DOWN = 2'd2;
11
12     reg [1:0] state;
13     reg [3:0] ptr;
14     reg [2:0] step_cnt;
15     reg [3:0] phase_cnt;
16
17     wire toggle_mode = (phase_cnt >= 4'd6);
18
19     always @(posedge clk or negedge rst_n) begin
20         if (!rst_n) begin
21             state      <= IDLE;
22             ptr         <= 4'd0;
23             step_cnt    <= 3'd0;
24             phase_cnt   <= 4'd0;
25             led_out     <= 16'b0;
```



```
26     end else begin
27         case (state)
28             IDLE: begin
29                 led_out    <= 16'b0;
30                 ptr        <= 4'd0;
31                 step_cnt   <= 3'd0;
32                 phase_cnt  <= 4'd0;
33                 if (start_sig) state <= UP;
34             end
35
36             UP: begin
37                 led_out[ptr] <= toggle_mode ? ~led_out[ptr] : 1'b1;
38                 if (step_cnt == 3'd7) begin
39                     step_cnt <= 3'd0;
40                     phase_cnt <= phase_cnt + 1'b1;
41                     state <= DOWN;
42                 end else begin
43                     ptr <= ptr + 1'b1;
44                     step_cnt <= step_cnt + 1'b1;
45                 end
46             end
47
48             DOWN: begin
49                 led_out[ptr] <= toggle_mode ? ~led_out[ptr] : 1'b0;
50                 if (step_cnt == 3'd3) begin
51                     step_cnt <= 3'd0;
52                     phase_cnt <= phase_cnt + 1'b1;
53                     if (toggle_mode &&
54                         ((led_out & ~(16'd1 << ptr)) == 16'b0)) begin
55                         led_out <= 16'b0;
56                         if (start_sig) begin
57                             state <= UP;
58                             ptr <= 4'd0;
59                             step_cnt <= 3'd0;
60                             phase_cnt <= 4'd0;
61                         end else
62                             state <= IDLE;
63                         end else
64                             state <= UP;
65                     end else begin
66                         ptr <= ptr - 1'b1;
67                         step_cnt <= step_cnt + 1'b1;
68                     end
69                 end
70                 default: state <= IDLE;
71             endcase
72         end
73     end
74 endmodule
```

4 Testbench

4.1 Tổng quan Testbench

Testbench được thiết kế theo cấu trúc **stimulus-driven**, tổ chức thành **4 test case** độc lập nối tiếp nhau trong một khối **initial** duy nhất. Mỗi test case kiểm tra một kịch bản hoạt động riêng biệt, đảm bảo phủ đầy đủ các trường hợp biên (*corner case*) của thiết kế, bao gồm hoạt động bình thường, restart và reset bất đồng bộ.

Testbench gồm ba thành phần chính:

- **Khai báo và khởi tạo:** Định nghĩa `timescale 1ns/1ps` để đảm bảo độ phân giải thời gian mô phỏng. Chu kỳ clock `CYCLE = 20ns` tương đương tần số **50 MHz**. DUT (`ring_flasher`) được instantiate với đầy đủ kết nối tín hiệu. Waveform được cấu hình dump tự động qua `$recordfile("waves")` và `$recordvars("depth=0")` để xuất toàn bộ tín hiệu phục vụ phân tích trên SimVision.
- **Clock generation:** Xung clock đối xứng 50% duty cycle được tạo bởi khối `always #(CYCLE/2) clk = ~clk`, khởi tạo `clk = 0` trong khối **initial** riêng biệt. Cách tổ chức này đảm bảo clock luôn ổn định trước khi stimulus bắt đầu tác động lên DUT.
- **Stimulus:** Khối **initial** chính áp dụng tuần tự các tổ hợp `rst_n` và `start_sig` theo từng test case. Kỹ thuật `@(posedge clk); #1;` được sử dụng nhất quán trước mỗi lần thay đổi tín hiệu — đây là thực hành chuẩn trong Verilog để đảm bảo stimulus được áp dụng sau cạnh clock đúng **1 ns**, tránh hoàn toàn race condition với logic DUT. Toàn bộ quá trình được ghi log qua `$display` với timestamp để hỗ trợ debug.

Bảng 8 tổng hợp 4 test case, kịch bản và mục tiêu kiểm tra tương ứng:

Test	Tên	Mục tiêu kiểm tra
TC1	1-clock pulse, về IDLE	Phát <code>start_sig</code> đúng 1 clock để kích hoạt FSM. Sau đó giữ <code>start_sig=0</code> xuyên suốt. Xác minh FSM tự vận hành hoàn chỉnh 1 vòng quét và về IDLE khi <i>all OFF</i> do <code>start_sig=0</code> .
TC2	Giữ HIGH 2 vòng, về IDLE	Giữ <code>start_sig=1</code> trong suốt 2 vòng quét liên tiếp, sau đó hạ xuống 0. Xác minh FSM hoàn thành vòng hiện tại rồi về IDLE tại <i>all OFF</i> tiếp theo.
TC3	Restart liên tục	Giữ <code>start_sig=1</code> xuyên suốt không giới hạn. Xác minh cơ chế restart: tại mỗi <i>all OFF</i> , FSM reset <code>ptr</code> và <code>phase_cnt</code> về 0 rồi bắt đầu vòng mới ngay, không qua IDLE.
TC4	Reset bất đồng bộ giữa chừng	Phát <code>rst_n=0</code> khi FSM đang hoạt động ở trạng thái UP. Xác minh FSM lập tức về IDLE, <code>led_out=0</code> , bất kể đang thực hiện bước nào, không chờ hoàn thành phase hiện tại.

Bảng 8: Các test case trong testbench

4.2 Testbench Code

Listing 1 trình bày toàn bộ mã nguồn testbench. Lưu ý kỹ thuật `@(posedge clk); #1;` được sử dụng nhất quán trước mỗi lần thay đổi tín hiệu - đây là thực hành chuẩn trong Verilog để đảm bảo stimulus được áp dụng sau cạnh clock một khoảng `#1`, tránh hoàn toàn race condition với logic DUT.

Listing 1: testbench.v

```
1  `timescale 1ns / 1ps
2  module testbench;
3      localparam CYCLE = 20;
4
5      reg  clk;
6      reg  rst_n;
7      reg  start_sig;
8      wire [15:0] led_out;
9
10     ring_flasher dut (
11         .clk      (clk),
12         .rst_n     (rst_n),
13         .start_sig (start_sig),
14         .led_out   (led_out)
15     );
16
17     initial clk = 1'b0;
18     always #(CYCLE/2) clk = ~clk;
19
20     always @(led_out)
21         $display("[%0t_uns] led_out = %016b", $time, led_out);
22
23     initial begin
24         $recordfile("waves");
25         $recordvars("depth=0");
26     end
27
28     initial begin
29         rst_n      = 1'b1;
30         start_sig = 1'b0;
31
32         // ===== RESET =====
33         $display("===== RESET =====");
34         @(posedge clk); #1;
35         rst_n = 1'b0;
36         repeat(3) @(posedge clk); #1;
37         rst_n = 1'b1;
38         $display("[%0t_uns] Reset released", $time);
39
40         // ===== TC1: 1-clock pulse -> IDLE =====
41         $display("===== TC1: 1-clock pulse =====");
42         repeat(2) @(posedge clk); #1;
43         start_sig = 1'b1;
```

```
44     $display("[%0t\ns]\tstart_sig=\t1", $time);
45     @(posedge clk); #1;
46     start_sig = 1'b0;
47     $display("[%0t\ns]\tstart_sig=\t0\t(1-clock_pulse)", $time);
48     repeat(287) @(posedge clk); #1;
49     $display("=====\tTC1:\tAll\tOFF\t->\tIDLE\t=====");
50     repeat(2) @(posedge clk); #1;
51
52     // ===== TC2: Giu HIGH -> IDLE =====
53     $display("=====\tTC2:\t2\tvong,\tve\tIDLE\t=====");
54     repeat(1) @(posedge clk); #1;
55     start_sig = 1'b1;
56     $display("[%0t\ns]\tstart_sig=\t1", $time);
57     repeat(288) @(posedge clk); #1;
58     start_sig = 1'b0;
59     $display("=====\tTC2:\tTat\tstart_sig->\tIDLE\t=====");
60     repeat(5) @(posedge clk); #1;
61
62     // ===== TC3: Restart =====
63     $display("=====\tTC3:\tRestart\t=====");
64     rst_n = 1'b0;
65     repeat(2) @(posedge clk); #1;
66     rst_n = 1'b1;
67     repeat(2) @(posedge clk); #1;
68     start_sig = 1'b1;
69     $display("[%0t\ns]\tRestart!", $time);
70     repeat(300) @(posedge clk); #1;
71     start_sig = 1'b0;
72     repeat(5) @(posedge clk); #1;
73     // ===== TC4: Reset giữa chung =====
74     $display("=====\tTC4:\tReset\tgiua\tchung\t=====");
75     rst_n = 1'b0;
76     repeat(2) @(posedge clk); #1;
77     rst_n = 1'b1;
78     repeat(2) @(posedge clk); #1;
79     start_sig = 1'b1;
80     repeat(20) @(posedge clk); #1;
81     $display("[%0t\ns]\tReset\tgiua\tchung!", $time);
82     rst_n = 1'b0;
83     repeat(2) @(posedge clk); #1;
84     rst_n = 1'b1;
85     repeat(5) @(posedge clk); #1;
86     start_sig = 1'b0;
87     repeat(5) @(posedge clk); #1;
88
89     $display("=====\tEND\tSIMULATION\t=====");
90     $finish;
91     end
92 endmodule
```

4.3 Chi tiết từng Test Case

Test Case 1 - 1-clock pulse, FSM tự chạy về IDLE

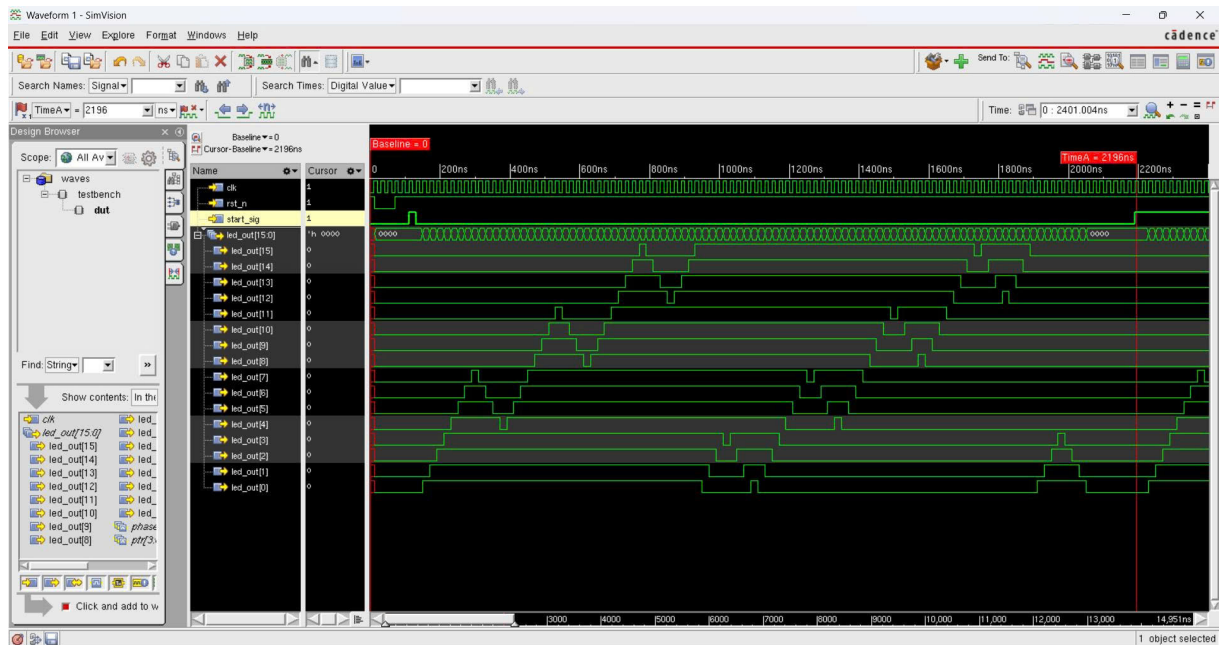
Mục tiêu: Xác minh `start_sig` hoạt động đúng như tín hiệu đơn xung. FSM chỉ nhận kích hoạt trong đúng 1 clock cycle, sau đó tự vận hành toàn bộ 12 phase mà không cần giữ tín hiệu điều khiển. Khi *all OFF*, do `start_sig` = 0, FSM về IDLE và dừng hoàn toàn.

Kịch bản thực thi:

- Phát `start_sig` = 1 đúng 1 clock cycle tại $t \approx 160$ ns, FSM chuyển từ IDLE sang UP.
- Hạ `start_sig` = 0 ngay clock tiếp theo và giữ nguyên suốt quá trình.
- FSM tự vận hành 12 phase (UP $\leftrightarrow$ DOWN) liên tục trong khoảng $t = 0 \rightarrow 14,951$ ns.
- Tại *all OFF*: `start_sig` = 0 $\rightarrow$ FSM về IDLE, `led_out` = 0x0000.

Luồng trạng thái kỳ vọng:

IDLE $\xrightarrow{\text{start_sig}=1 \text{ (1 clock)}}$ UP $\leftrightarrow$ DOWN $\times 12$ phase $\xrightarrow{\text{all OFF, start_sig}=0}$ IDLE



Hình 2: Waveform TC1 - 1-clock pulse, FSM tự chạy và về IDLE

Nhận xét: Waveform cho thấy `start_sig` chỉ lên HIGH trong một khoảng rất ngắn ở đầu timeline (xung nhỏ tại $t \approx 160$ ns). Sau đó các tín hiệu `led_out[0]` đến `led_out[15]` lần lượt bật/tắt theo đúng chuỗi quét Normal rồi Toggle mode. Cuối timeline, `led_out` về 0x0000 và FSM dừng hẳn tại IDLE.

Test Case 2 - Giữ `start_sig` HIGH, về IDLE

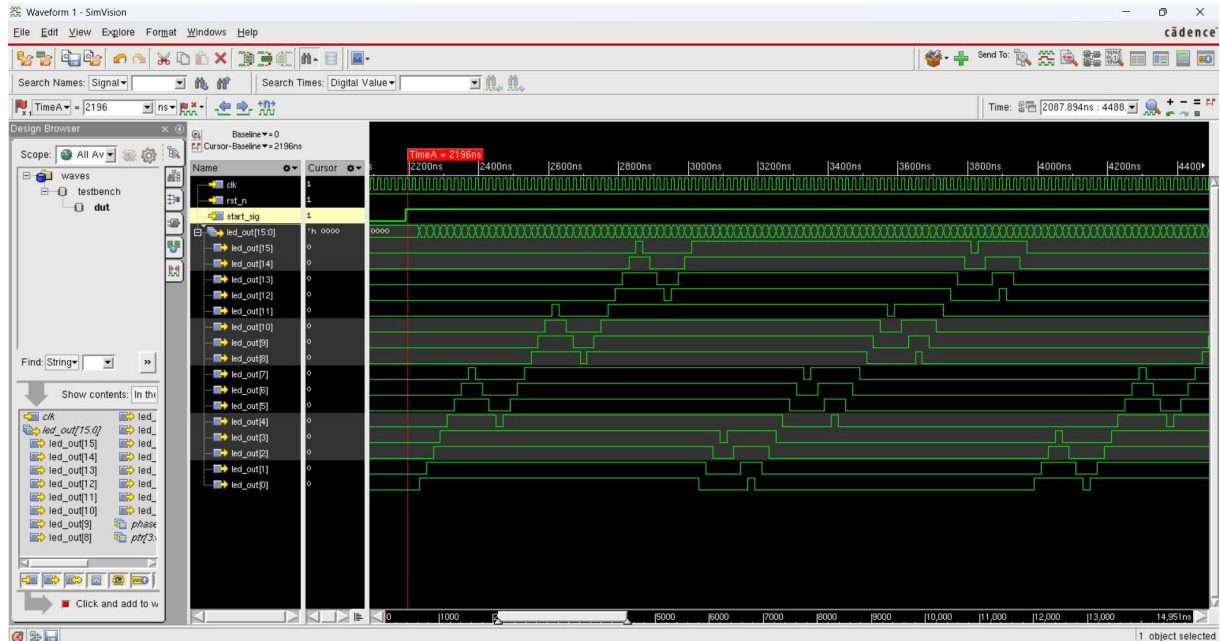
Mục tiêu: Kiểm tra hành vi FSM khi `start_sig` được giữ HIGH liên tục qua 2 vòng quét. Xác minh FSM chỉ sample `start_sig` tại đúng thời điểm *all OFF* để quyết định restart hay dừng, và không bị ảnh hưởng bởi mức tín hiệu trong khi đang chạy.

Kịch bản thực thi:

- Giữ `start_sig` = 1 liên tục từ $t \approx 220$ ns.
- FSM hoàn thành vòng 1: tại *all OFF* đầu tiên, `start_sig` = 1 $\rightarrow$ restart ngay, bắt đầu vòng 2.
- Hạ `start_sig` = 0 tại $t \approx 2196$ ns (giữa chừng vòng 2).
- FSM hoàn thành vòng 2: tại *all OFF* tiếp theo, `start_sig` = 0 $\rightarrow$ về IDLE.

Luồng trạng thái kỳ vọng:

IDLE $\xrightarrow{\text{start_sig}=1}$ Vòng 1 $\xrightarrow{\text{all OFF, start_sig}=1}$ Vòng 2 $\xrightarrow{\text{all OFF, start_sig}=0}$ IDLE



Hình 3: Waveform TC2 - Giữ `start_sig` HIGH 2 vòng, về IDLE

Nhận xét: Waveform cho thấy `start_sig` giữ HIGH từ đầu đến $t \approx 2196$ ns (cursor đỏ). Hai vòng quét liên tiếp được quan sát rõ ràng qua pattern bật/tắt lặp lại của `led_out[0..15]`. Sau khi `start_sig` hạ xuống 0, FSM hoàn thành vòng 2 rồi về IDLE với `led_out` = 0x0000.

Test Case 3 - Restart liên tục

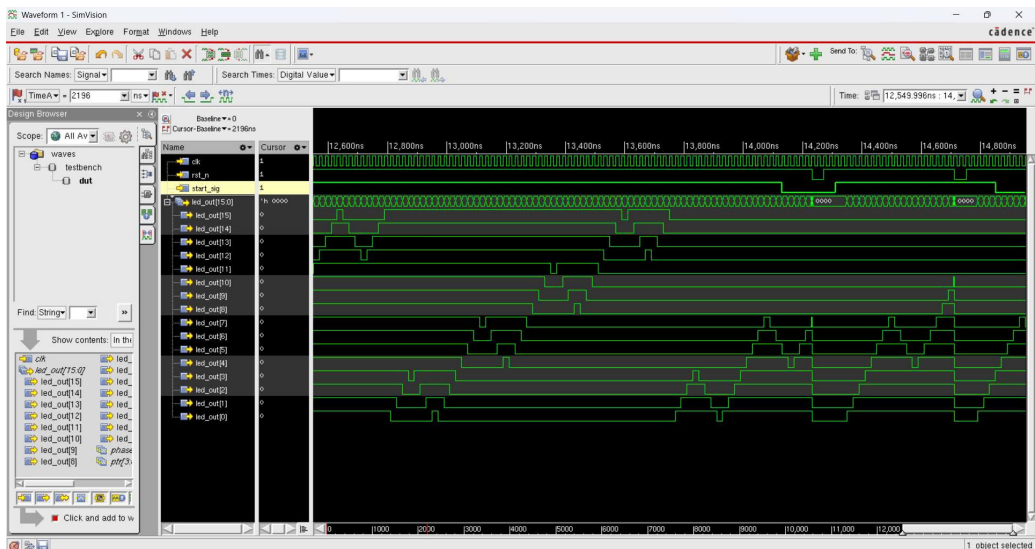
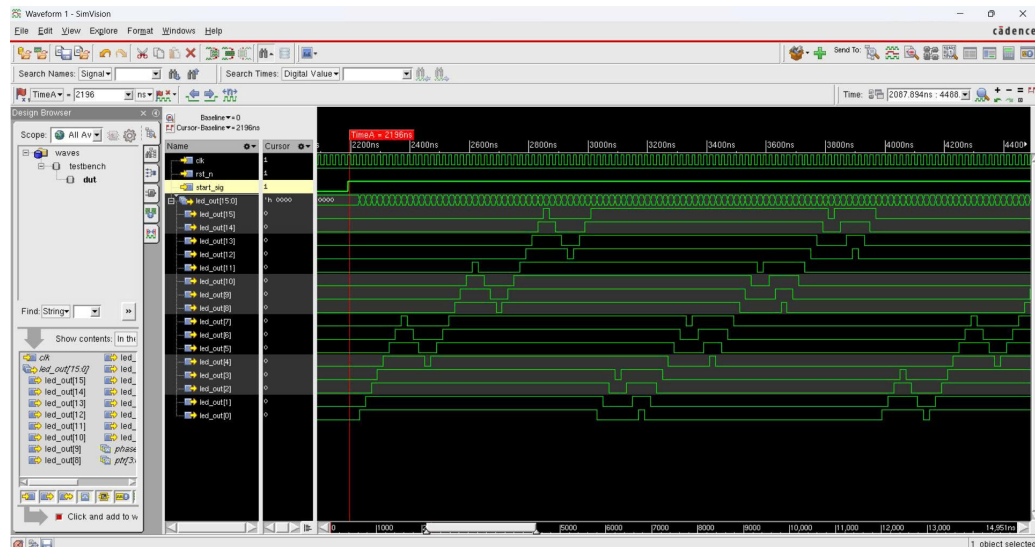
Mục tiêu: Kiểm tra cơ chế restart của FSM khi `start_sig` giữ HIGH xuyên suốt không giới hạn. Xác minh tại mỗi *all OFF*, FSM reset đúng `ptr = 0` và `phase_cnt = 0`, bắt đầu vòng mới ngay lập tức mà không qua IDLE.

Kịch bản thực thi:

- Giữ `start_sig = 1` liên tục xuyên suốt toàn bộ quá trình mô phỏng.
- Tại mỗi *all OFF* (quan sát tại $t \approx 12,600$ ns và $t \approx 14,600$ ns): `led_out` về 0x0000 tức thì rồi bắt đầu chuỗi quét mới ngay clock tiếp theo.
- FSM không đi qua IDLE giữa các vòng.

Luồng trạng thái kỳ vọng:

... Vòng n $\xrightarrow{\text{all OFF, start_sig=1}}$ Vòng n+1 $\xrightarrow{\text{all OFF, start_sig=1}}$ Vòng n+2 ...



Hình 4: Waveform TC3 - Restart liên tục khi `start_sig` giữ HIGH

Test Case 4 - Reset bất đồng bộ giữa chừng

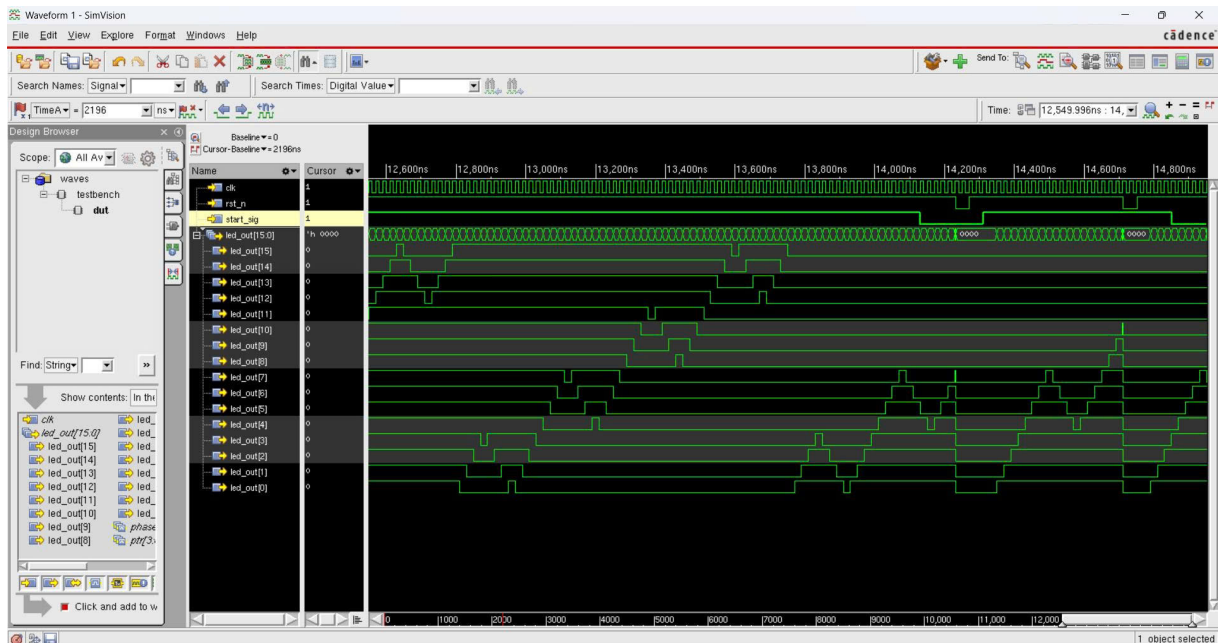
Mục tiêu: Kiểm tra tính đúng đắn của cơ chế reset bất đồng bộ (*asynchronous reset*). Phát $\text{rst_n} = 0$ khi FSM đang hoạt động ở trạng thái UP, xác minh rằng FSM lập tức về IDLE và xóa toàn bộ LED - không chờ hoàn thành bước hay phase hiện tại, không phụ thuộc vào cạnh clock.

Kịch bản thực thi:

- Kích hoạt FSM bằng $\text{start_sig} = 1$, để FSM chạy vào trạng thái UP sau khoảng 20 clock cycle.
- Phát $\text{rst_n} = 0$ đột ngột giữa chừng khi FSM đang ở UP, chưa hoàn thành phase.
- Giữ $\text{rst_n} = 0$ trong 2 clock, sau đó thả ra $\text{rst_n} = 1$.
- Xác minh $\text{led_out} = 0x0000$ và FSM ở IDLE ngay tại thời điểm rst_n xuống 0, không trễ so với clock.

Luồng trạng thái kỳ vọng:

IDLE $\xrightarrow{\text{start_sig}=1}$ UP (*đang chạy*) $\xrightarrow{\text{rst_n}=0 \text{ (async)}}$ IDLE (*led_out=0, tức thì*)



Hình 5: Waveform TC4 - Reset bất đồng bộ giữa chừng khi FSM đang ở UP

Nhận xét: Waveform xác nhận led_out về $0x0000$ ngay tại cạnh xuống của rst_n , không trễ so với clock. Toàn bộ các bit $\text{led_out}[15:0]$ hạ về 0 đồng thời, FSM trở về IDLE tức thì bất kể đang thực hiện bước nào trong phase UP. Sau khi rst_n được thả ra, FSM ở lại IDLE ổn định cho đến khi có tác động tiếp theo. Đúng với đặc tả reset bất đồng bộ tích cực thấp.

5 Kết luận

5.1 Kết quả đạt được

Toàn bộ 4 test case đã được thực thi thành công trên môi trường mô phỏng Cadence Xcelium. Waveform thu được từ SimVision xác nhận thiết kế hoạt động đúng theo đặc tả ở mọi kịch bản kiểm tra, bao gồm cả các trường hợp biên (*corner case*) như reset bất đồng bộ và restart liên tục.

TC	Kịch bản	Kết quả	Nhận xét
TC1	1-clock pulse, về IDLE	PASS	FSM tự hoàn thành đủ 12 phase và về IDLE tại <i>all OFF</i> khi <i>start_sig</i> =0.
TC2	Giữ HIGH 2 vòng, về IDLE	PASS	FSM chạy đúng 2 vòng liên tiếp; về IDLE tại <i>all OFF</i> vòng 2 khi <i>start_sig</i> đã hạ.
TC3	Restart liên tục	PASS	<i>ptr</i> và <i>phase_cnt</i> reset đúng về 0 tại mỗi <i>all OFF</i> ; không qua IDLE giữa các vòng.
TC4	Reset bất đồng bộ giữa chừng	PASS	<i>led_out</i> tức thì khi <i>rst_n</i> =0, không chờ cạnh clock hay hoàn thành bước hiện tại.

Bảng 9: Tổng hợp kết quả các test case

5.2 Những điểm đã học được

- **Hiện thực FSM một khối always:** Nắm vững cách tổ chức toàn bộ logic trạng thái, con trỏ và bộ đếm trong một khối đồng bộ duy nhất. Cách tiếp cận này tránh hoàn toàn latch inference và phù hợp với quy trình tổng hợp (*synthesis*) công nghiệp.
- **Phân biệt tín hiệu level và tín hiệu pulse:** Hiểu rõ sự khác biệt quan trọng giữa tín hiệu mức (*level*) và tín hiệu xung (*pulse*) trong thiết kế FSM. *start_sig* chỉ có ý nghĩa tại đúng 2 thời điểm xác định; ở các thời điểm khác FSM hoàn toàn bỏ qua giá trị của tín hiệu này.
- **Toggle mode điều khiển bằng bộ đếm pha:** Nắm được kỹ thuật thay đổi hành vi đầu ra theo *phase_cnt* thông qua wire tổ hợp *toggle_mode*, không cần bổ sung trạng thái FSM mới - giữ thiết kế đơn giản và tiết kiệm tài nguyên.
- **Reset bất đồng bộ trong Verilog:** Hiểu cơ chế *negedge rst_n* trong sensitivity list của khối *always*, đảm bảo reset có hiệu lực tức thì độc lập với clock, đúng với yêu cầu thiết kế số thực tế.
- **Phân tích waveform bằng SimVision:** Thành thạo sử dụng *Source Browser* để theo dõi giá trị thanh ghi theo từng clock cycle, kết hợp tính năng zoom và *Signal Tracer* để debug và xác minh hành vi FSM một cách trực quan và chính xác.